



Alumno—

Hector Javier Alonso Zaragoza

ID—

00000252039

Asignación—

Escrito sobre implementación de Casos de Uso

Materia—

Arquitectura de Software

Durante el desarrollo del juego UNO, la separación de los casos de uso entre los compañeros de equipo me dejó con los casos de Abandonar Partida y Solicitar Iniciar Partida. Estos ponen a prueba el funcionamiento de la arquitectura dirigida por eventos y la comunicación efectiva entre los peers de la red.

En el contexto de Abandonar Partida, cualquier jugador puede presionar el botón de salir y confirmar su decisión. Si quien abandona es un Peer, la partida continúa para los demás; si es el Host, todos los jugadores regresan al menú principal. Internamente, la acción comienza en la GameView, que delega la lógica al GameController. A partir de ahí, el comportamiento cambia según el rol del jugador. Cuando un Peer sale, se publica un PlayerDisconnectedEvent en el EventBus, y la NetworkLayer se encarga de notificar al Host mediante sockets. El Host actualiza el estado de la partida, elimina al jugador y sincroniza los cambios con los demás clientes. Si el que abandona es el Host, se envía un mensaje a todos los Peers para cerrar la partida inmediatamente.

Sobre la Solicitud para Iniciar Partida, un Peer que ya marcó “Estoy Listo” puede pedirle al Host comenzar la partida aunque aún no estén todos los jugadores conectados. La solicitud nace en LobbyView, pasa por LobbyController y se publica como un StartRequestedEvent. Después, la NetworkLayer envía la petición al Host, quien recibe una ventana de confirmación para aceptar o rechazar el inicio. Si acepta, se valida el estado del lobby, se publica un GameStartedEvent y todos los jugadores son enviados a la pantalla del juego.

Aplicando el patrón Modelo-Vista-Controlador, separamos la lógica de los datos y la interfaz gráfica de usuario. Con este patrón delegamos la lógica a una clase controladora mientras la interfaz se limitaba solo a mostrar datos y recibir el input del jugador. Observer implementado sobre el EventBus evita el acoplamiento entre los

componentes de modo que quienes reaccionan a eventos publicados no necesitan de la lógica del publicador.

Como conclusión, el proyecto dejó clara la importancia de manejar correctamente los listeners del EventBus y de mantener al Host como única fuente de verdad para evitar inconsistencias entre clientes.